eclipup.in

RISC-V AXI Integration

Course: PicoRV32 SoC Subsystem with AXI-Lite & UART Integration

1. Title

RISC-V AXI Integration — Bridging PicoRV32 Native Bus to AXI-Lite Protocol

2. Objective

By the end of this module, learners will be able to:

- Explain why a protocol bridge is needed between PicoRV32 and AXI peripherals
- Describe the CPU-to-AXI bridge state machine implemented in `rtl/top.v`
- Trace an AXI-Lite write/read transaction generated by the PicoRV32 native bus
- Run the `make riscv_axi` demo testbench and observe formatted handshake logs
- Identify the FSM states: `ST_IDLE → ST_WR_AW → ST_WR_B → ST_RD_AR → ST_RD_R`

3. Theory (Multi-Level)

3.1 Beginner — Simple Intuition

PicoRV32 speaks its own simple language: "here is an address and data, tell me when you are done." The AXI peripheral speaks a more formal protocol: five separate conversations (channels) with specific handshake rules.

The **AXI bridge** is a translator that stands between them:

```
PicoRV32          AXI Bridge              AXI Slave
(native bus)   (protocol converter)      (ROM/SRAM/UART)

mem_valid=1  ──►  translates to ──►   awvalid=1 (write addr)
                                    + wvalid=1  (write data)
                  waits for         ◄── bvalid=1 (response)
mem_ready=1  ◄──  signals done  ──────────────────
```

The CPU never "sees" AXI at all — it just uses its simple `mem_valid/mem_ready` protocol, and the bridge handles all the AXI complexity underneath.

3.2 Intermediate — Engineering View

Why is the bridge needed?

PicoRV32's native memory interface is a **single channel** with one valid/ready pair. AXI-Lite has **five independent channels**. A direct connection is not possible.

The bridge must: 1. Detect if the CPU wants a read (wstrb=0) or write (wstrb≠0) 2. Launch the appropriate AXI channel(s) 3. Wait for AXI completion across multiple clock cycles 4. Assert `mem_ready` when the AXI transaction finishes 5. Return `mem_rdata` for reads

FSM States (from `rtl/top.v`):

```
ST_IDLE   (3'd0): Wait for cpu_mem_valid
ST_WR_AW  (3'd1): Issue AW+W channels, wait for B response
ST_WR_B   (3'd2): Wait for B channel (if AW/W already done)
ST_RD_AR  (3'd3): Issue AR channel, wait for arready
ST_RD_R   (3'd4): Wait for R channel data
```

3.3 Advanced — Signal/Architecture Level

Bridge internal signal mapping (`rtl/top.v`, lines 102–224):

```
// CPU outputs (inputs to bridge)
wire cpu_mem_valid;       // CPU has a pending request
wire [31:0] cpu_mem_addr;
wire [31:0] cpu_mem_wdata;
wire [3:0]  cpu_mem_wstrb;

// Bridge outputs (to CPU)
reg  ready_r;             // → cpu_mem_ready  (one-cycle pulse)
reg  [31:0] rdata_r;      // → cpu_mem_rdata  (latched read data)

assign cpu_mem_ready = ready_r;
assign cpu_mem_rdata = rdata_r;
```

Write Transaction FSM Detail:

```
ST_IDLE:
  if (cpu_mem_valid && |cpu_mem_wstrb):
    m_axi_awaddr  ← cpu_mem_addr
    m_axi_awvalid ← 1
    m_axi_wdata   ← cpu_mem_wdata
    m_axi_wstrb_r ← cpu_mem_wstrb
    m_axi_wvalid  ← 1
    → ST_WR_AW

ST_WR_AW:
  if (m_axi_awready && m_axi_awvalid): m_axi_awvalid ← 0
  if (m_axi_wready  && m_axi_wvalid):  m_axi_wvalid  ← 0
  m_axi_bready ← 1
  if (m_axi_bvalid && m_axi_bready):   ready_r ← 1 → ST_IDLE
  elif (!m_axi_awvalid && !m_axi_wvalid): → ST_WR_B

ST_WR_B:
  if (m_axi_bvalid): ready_r ← 1 → ST_IDLE
```

Read Transaction FSM Detail:

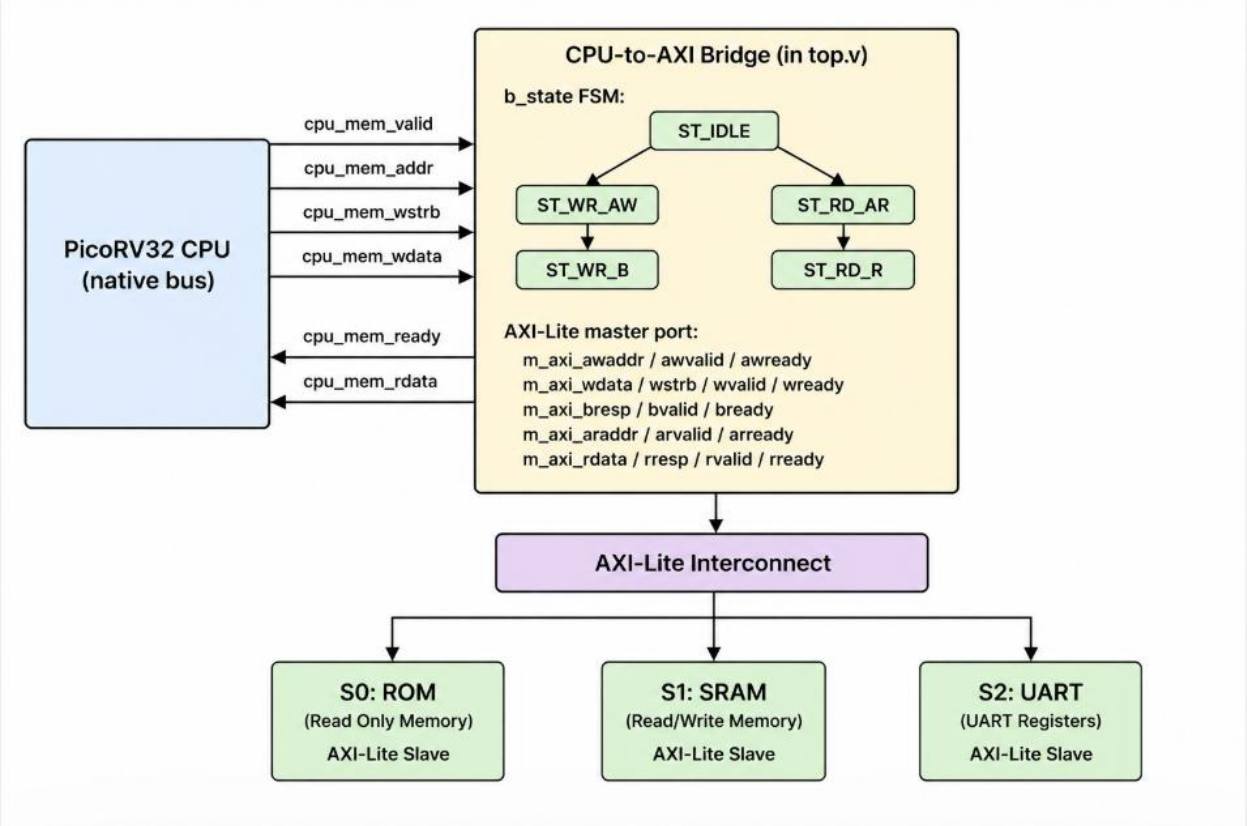
```
ST_IDLE:
  if (cpu_mem_valid && !|cpu_mem_wstrb):
    m_axi_araddr  ← cpu_mem_addr
    m_axi_arvalid ← 1
    → ST_RD_AR

ST_RD_AR:
  if (m_axi_arready && m_axi_arvalid):
    m_axi_arvalid ← 0
    m_axi_rready  ← 1
    → ST_RD_R

ST_RD_R:
  if (m_axi_rvalid && m_axi_rready):
    rdata_r      ← m_axi_rdata
    m_axi_rready ← 0
    ready_r      ← 1
    → ST_IDLE
```

4. Architecture / Concept

Full Bridge Architecture



Timing: CPU to AXI Write (Best Case)

```
Cycle:    1          2          3          4          5
CPU:      valid=1    valid=1    valid=1    ready=0    mem_ready=1
Bridge:   ST_IDLE    ST_WR_AW   (↓AW↓W)   ST_WR_B    ST_IDLE
AXI:      awvalid=1  awvalid=1  awready=1  bvalid=1   bready=1
          wvalid=1   wvalid=1   wready=1
```

The bridge introduces 2–4 cycle latency per AXI transaction, which is acceptable for MMIO register access.

5. Implementation

Files in This Module

File	Role
`rtl/top.v`	Contains the CPU-to-AXI bridge FSM (lines 91–224)
`tb/tb_riscv_axi_lite.v`	**Self-contained** demo TB showing step-by-step AXI handshake
`rtl/axi_lite_interconnect.v`	AXI interconnect that bridge connects to
`rtl/axi_decoder.v`	Address decoder used by interconnect

Bridge Registers (from `rtl/top.v`)

```
// ASIC-clean: no inline initializers — all driven by reset
reg  [31:0] m_axi_awaddr;    reg m_axi_awvalid;
reg  [31:0] m_axi_wdata;     reg [3:0] m_axi_wstrb_r; reg m_axi_wvalid;
reg         m_axi_bready;
reg  [31:0] m_axi_araddr;    reg m_axi_arvalid;
reg         m_axi_rready;
reg [2:0]   b_state;
reg [31:0]  rdata_r;
reg         ready_r;
```

Demo Testbench (`tb_riscv_axi_lite.v`) — What It Does

This testbench simulates a CPU issuing an AXI write then read at address `0x1000_0000` (UART base): 1. Master asserts `m_awaddr=0x10000000`, `m_awvalid=1`, `m_wdata=0xA5`, `m_wvalid=1` 2. Slave responds: `s_awready=1`, `s_wready=1` → write data stored in `slave_reg[0]` 3. Slave asserts `s_bvalid=1` (OKAY response) 4. Master reads back: `m_arvalid=1` → `s_arready=1` → `s_rdata=0xA5` → `s_rvalid=1` 5. Checks `captured_rdata == TEST_DATA` → prints “Success”

6. Lab / Hands-on Section

Step 1: Run the RISC-V AXI Integration Demo

```
cd ~/Desktop/OpenLaneUser/designs/picorv32_soc
make riscv_axi
```

This: 1. Cleans `obj_axi/` 2. Compiles only `tb/tb_riscv_axi_lite.v` (self-contained, no RTL files needed) 3. Runs `./obj_axi/sim_axi` → produces `tb_riscv_axi_lite.vcd` 4. Opens GTKWave

Expected Terminal Output

```
---- Simulation Start ----

[0-10 ns] System in reset
[10 ns] Reset released → CPU and AXI start working

------------------------------
[50 ns] AXI WRITE TRANSACTION
------------------------------
CPU wants to write data

Step 1: Send Address
  AWADDR  = 0x10000000
  AWVALID = 1  → CPU says "address is ready"
  AWREADY = 1  → Bus says "address accepted"

Step 2: Send Data
  WDATA   = 0x000000A5
  WVALID  = 1  → CPU says "data is ready"
  WREADY  = 1  → Bus says "data accepted"

Step 3: Write Response
  BVALID  = 1  → Bus says "write completed"
  BREADY  = 1  → CPU accepts response

 Result: Data 0xA5 successfully written to address 0x10000000

------------------------------
[110 ns] AXI READ TRANSACTION
------------------------------
CPU wants to read data

Step 1: Send Address
  ARADDR  = 0x10000000
  ARVALID = 1  → CPU requests read
  ARREADY = 1  → Bus accepts request

Step 2: Get Data
```

eGurukul.in

```
  RDATA   = 0x000000A5
  RVALID  = 1  → Bus sends data
  RREADY  = 1  → CPU accepts data

 Result: Read value = 0xA5 (same as written)

-------------------------------
Final Conclusion:
Write = Success
Read  = Success
AXI-Lite integration is working correctly

---- Simulation End ----
```

Step 2: Trace the AXI Write Handshake in GTKWave

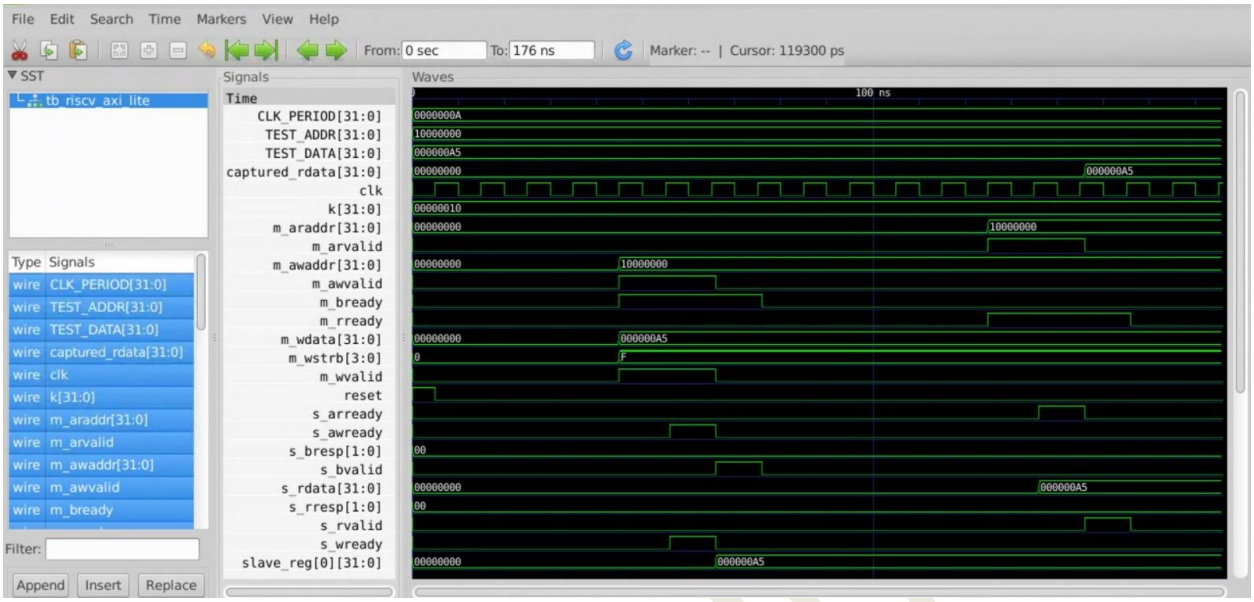
```
gtkwave tb_riscv_axi_lite.vcd &
```

Add:

```
clk
reset
m_awaddr[31:0]    ← master write address
m_awvalid
s_awready         ← slave accepts address
m_wdata[31:0]     ← master write data
m_wvalid
s_wready          ← slave accepts data
s_bvalid          ← slave response
m_bready          ← master ready for response
m_araddr[31:0]    ← master read address
m_arvalid
s_arready         ← slave accepts address
s_rdata[31:0]     ← slave read data
s_rvalid          ← slave data valid
m_rready          ← master ready for data
```

edhub.in

7. Waveform / Verification Insight



Write Transaction Checkpoint

In GTKWave, look for the cycle where `m_awvalid AND s_awready = 1` simultaneously — this is the AW handshake. Measure the delay from `m_awvalid` rising to `s_awready` rising. In this testbench it is 1 cycle (ideal slave).

Read Transaction Checkpoint

Look for `m_arvalid AND s_arready = 1` → AR handshake. Then `s_rvalid AND m_rready = 1` → R handshake. Confirm `s_rdata = 0xA5` at the R handshake moment.

Bridge FSM in Real SoC (tb_top.v context)

When running `make soc`, add `b_state[2:0]` to see the bridge FSM:

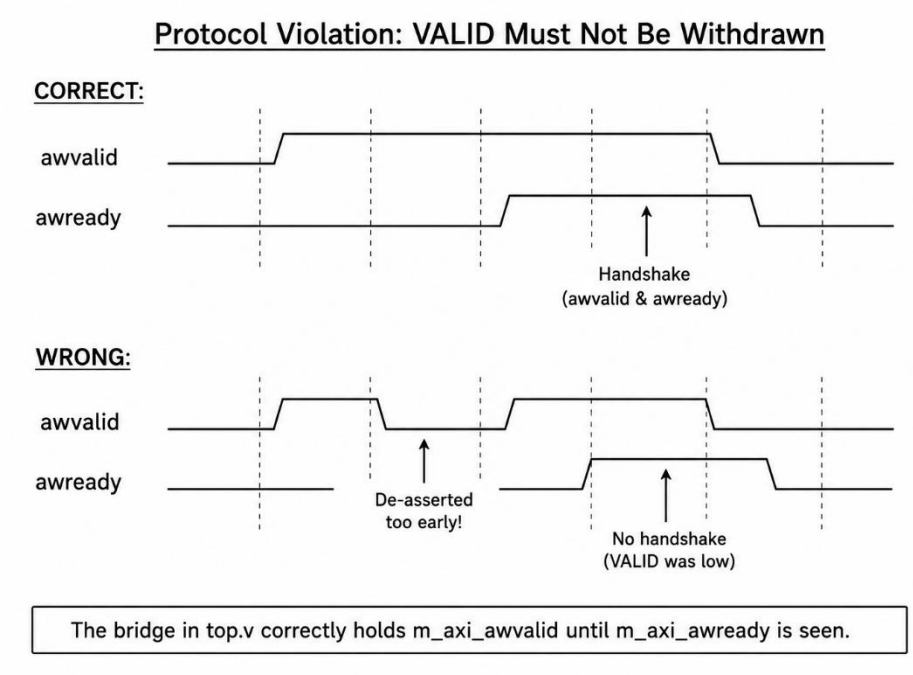
```
State value   Meaning
0 (ST_IDLE)   Waiting for cpu_mem_valid
1 (ST_WR_AW)  AW+W presented to interconnect
2 (ST_WR_B)   Waiting for B (write response)
3 (ST_RD_AR)  AR presented to interconnect
4 (ST_RD_R)   Waiting for R (read data)
```

The transition `0→1→0` or `0→3→4→0` visible in waveforms corresponds directly to each CPU instruction that accesses memory.

8. Common Issues & Debug Tips

Issue	Likely Cause	Fix
Bridge stuck in ST_WR_AW	`m_axi_awready` never asserts	Check interconnect routing — is wr_sel correct?
Bridge stuck in ST_RD_R	`m_axi_rvalid` never asserts	Check slave's read path — is `r_valid_r` never set?
`mem_ready` never pulses	Bridge FSM never reaches `ready_r ← 1`	Trace `b_state` in waveforms
CPU sees wrong rdata	`rdata_r` captured at wrong cycle	Ensure `rdata_r ← m_axi_rdata` happens only when `rvalid=1 AND rready=1`
Bridge issues both AW and AR	CPU wstrb ambiguous	Confirm decoder uses `|cpu_mem_wstrb != 0` → write
GTKWave shows no transitions	Wrong VCD file	Use `ls -la *.vcd` to verify `tb_riscv_axi_lite.vcd` was generated

Protocol Violation: VALID Must Not Be Withdrawn



9. Summary

Concept	Detail
Bridge purpose	Converts PicoRV32 native (`mem_valid/ready`) to AXI-Lite 5-channel protocol
FSM states	ST_IDLE, ST_WR_AW, ST_WR_B, ST_RD_AR, ST_RD_R
Write latency	3–4 cycles (AW+W handshake + B response)
Read latency	2–3 cycles (AR handshake + R data)
Bridge location	Embedded in `rtl/top.v` (lines 91–224)
ASIC-clean	All bridge registers reset-driven; no inline initializers
Lab file	`tb/tb_riscv_axi_lite.v` — fully self-contained, no RTL dependencies
Lab result	Write 0xA5 to 0x10000000, read back 0xA5 → Success

This module forms the critical connection between the RISC-V ISA world (Sub-modules 1–4) and the AXI peripheral world (Sub-modules 6–8). Every firmware memory access in the final SoC passes through this bridge.

Previous: *Sub-module 4 — MMI RISC-V + UART*
Next: *Sub-module 6 — AXI + UART Integration*

eInfochips